TIC-TAC-TOE AI

Python Project Report

An interactive two-player-style game in which a human player competes against an AI opponent using the Minimax decision-making algorithm.

Project	Tic-Tac-Toe AI
Programming language	Python
Development environment	Google Colab
AI technique	Minimax algorithm
Player symbols	Human: X | AI: O

Prepared as a project documentation draft. Add your name, college details and saved output screenshot before submission.

1. Introduction

Tic-Tac-Toe is a classic strategy game played on a 3 × 3 board. Two sides take turns placing their symbols in empty cells. The first side to form a complete row, column or diagonal wins. If all nine cells are filled without a winner, the game ends in a tie.

This project implements a human-versus-computer version in Python. The human plays as X and the AI plays as O. The board is displayed in the notebook output, and the player enters a cell position from 0 to 8.

2. Project Objectives

- Build an interactive Tic-Tac-Toe game using Python.
- Accept and validate the human player's moves.
- Detect wins, losses and ties.
- Implement an AI opponent that evaluates possible future moves using Minimax.
- Test the game through a complete match in Google Colab.

3. Tools and Technologies

Tool / technology	Purpose
Python	Core programming language
Google Colab	Cloud notebook used to write and run the code
random module	Used in the earlier basic/random AI test cell
Minimax algorithm	Evaluates possible game outcomes to select an AI move

4. Game Design and Working

Board representation: The board is stored as a list of nine cells. An empty cell is represented by a blank string, while X and O represent the two players.

Human move: The program asks the user to enter a position from 0 to 8. It checks that the input is valid and that the selected cell is empty.

Winner detection: The program checks the eight possible winning lines: three rows, three columns and two diagonals. It also checks whether the board is full to identify a tie.

AI move: The Minimax function explores available moves recursively. It assigns a positive score to an AI win, a negative score to a human win, and zero to a tie. The AI chooses the move with the best score, assuming both sides play optimally.

5. Minimax Algorithm

Minimax is a decision-making algorithm commonly used in turn-based games. It considers possible future moves and chooses the move that gives the AI the best outcome, while assuming the opponent will also choose strong moves.

In this project, O is the maximizing player and X is the minimizing player. The algorithm evaluates terminal game states and works backward through possible moves to choose an AI action.

Simplified scoring used in the implementation:

```
AI win (O) = +1
Human win (X) = -1
```

```
Tie = 0
```

6. Testing and Output

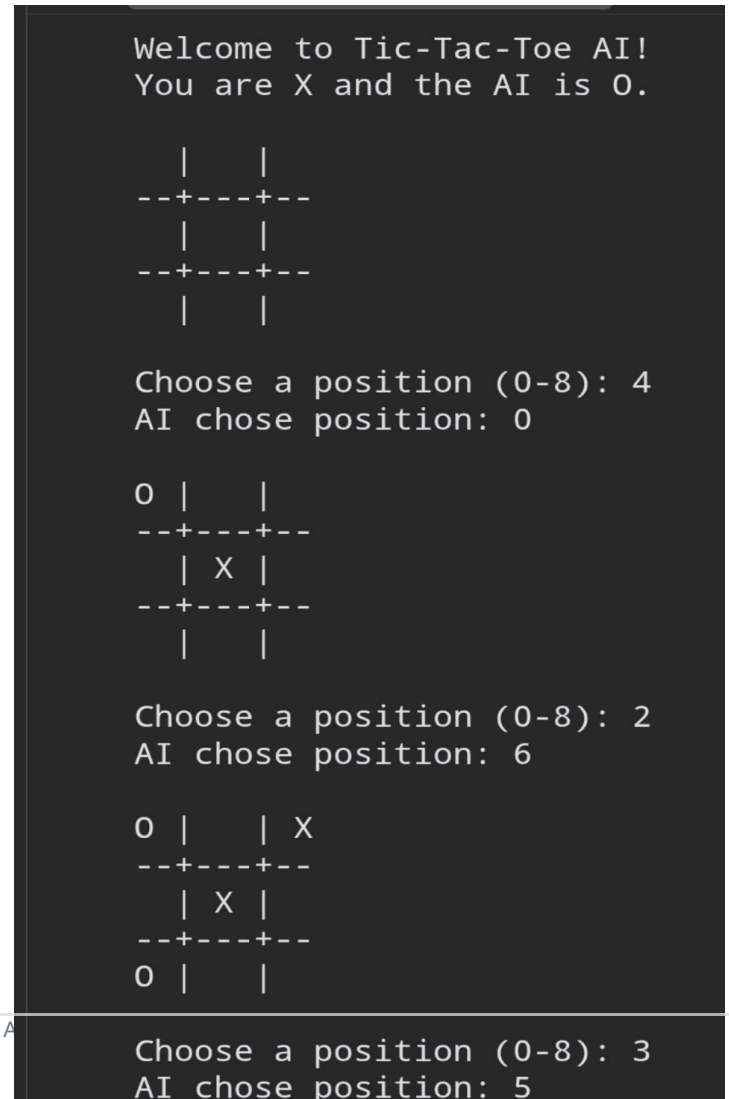
The game was run in Google Colab and a complete match was played. The recorded test sequence was:

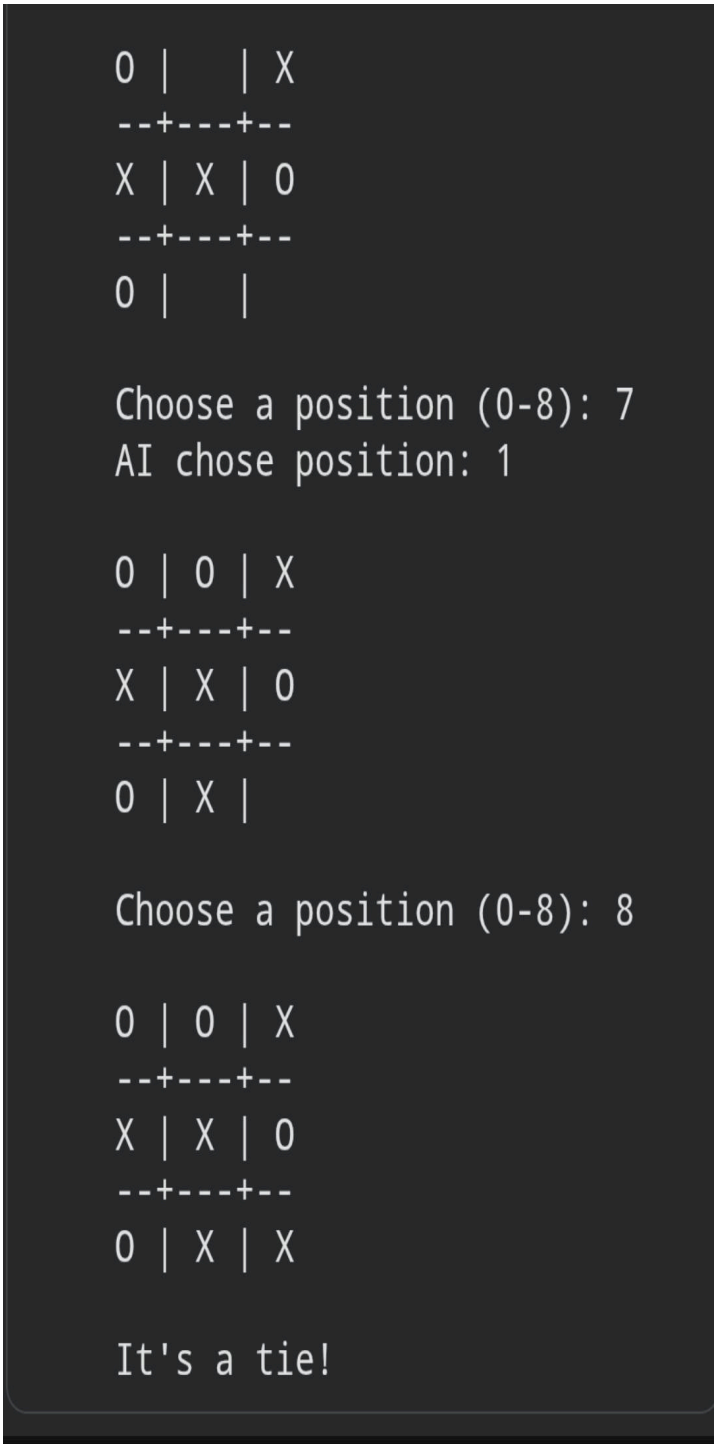
Turn	Player	Position
1	Human (X)	4
2	AI (O)	0
3	Human (X)	2
4	AI (O)	6
5	Human (X)	3
6	AI (O)	5
7	Human (X)	7
8	AI (O)	1
9	Human (X)	8

Final board from the recorded test:

```
O | O | X
X | X | O
O | X | X
```

Observed result: It's a tie! The game completed without a crash and correctly reported a tie for this test match.




O | | X
--+---+--
X | X | O
--+---+--
O | |
Choose a position (0-8): 7
AI chose position: 1
O | O | X
--+---+--
X | X | O
--+---+--
O | X |
Choose a position (0-8): 8
O | O | X
--+---+--
X | X | O
--+---+--
O | X | X
It's a tie!

Paste your saved final Google Colab output screenshot here when editing the report. The screenshot should show the final board and the message “It’s a tie!”.Screenshot to insert

7. Conclusion

The Tic-Tac-Toe AI project demonstrates how Python can be used to create an interactive game and how the Minimax algorithm can support strategic decision-making. In the recorded test, the AI responded to the human player's moves and the match ended in a tie. This confirms that the tested game flow, move handling and tie detection worked for this match. Further testing across different move sequences is recommended before describing the AI as unbeatable.

8. Future Enhancements

- Add a graphical user interface.
- Add a restart or play-again option.
- Test multiple game sequences, including corner, edge and center openings.
- Improve the interface with clearer position labels and instructions.

Student name: _J.karthik Nandu____

College / department: _Uttaranchal University / CSE-AIML

Date: 30/10/3026.